

Note: In one class file two class can not be public.

URBAN
EDGE

Concepts which are left

In Java

- (1) static member variable
- (2) static member function
- (3) we can not make static variable inside method
- (4) But we can make static inner class.

Code

```
public class Example  
{
```

```
    int n; // Instance variable
```

```
    static int y; // static member variable
```

It
can only
be
called
using
object
of that
class.

```
    public void fun1() { } // instance member method  
    public static void fun2() { } // static member method
```

```
    public void fun2  
    {
```

```
        static int z;
```

```
    }
```

~~we cannot make static variable inside class.~~

```
static class test ✓
```

```
{
```

```
}
```

3

static variables

- (1) static variables are declared in the class using static keyword.
- (2) static variables are by default initialized to its default value.
- (3) static member has a single copy for the whole class & does not depends on the objects.

Let us understand 2 & 3 point with example :

```
public class Example  
{  
    int x;  
    static int y;           // static member variable
```

```
public static void main(String[] args)  
{
```

```
    Example ex1 = new Example();
```

```
    Example ex2 = new Example();
```

```
    Example.y = 5;           → Now its default value
```

↙ static variable y is a
single copy of class

Every
time
variable
x will
be
created

changed

3

3

static method

- (1) static methods defined inside the class are qualified with the keyword static.
- (2) static methods can only access static members of the same class.
- (3) static methods can be invoked using class name & dot operator.

Real Life Examples of static concepts

Account Number, Balance = ~~static~~ instance variable

Rate of Interest = static variables.

Code

```
class Example
```

```
{
```

```
    int x;
```

```
    static int y;
```

```
    private static int z;
```

```
    public void fun1()
```

```
    {
```

```
        x = 4; ✓
```

```
    }
```

```
    public void static void fun2()
```

```
    {
```

```
        // x = 4; X
```

Inside static method
we cannot access
instance variable

```
        y = 4; ✓
```

```
        z = 4; ✓
```

```
    }
```

we can access
static variable

```
    public static void main(String args[])
```

```
    {
```

```
        y = 4; ✓
```

```
        z = 4; ✓
```

```
        // fun1(); X
```

```
        fun2(); ✓
```

```
    }
```

```
}
```

```
public class Hello
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

// y = 4; X

Example.y = 4; ✓

// z = 4; X

// Example.z = 4; X

But we can
access z variable
using method

// fun2(); X

Example.fun2(); ✓

Now here
z value will
become 4;

3

3

Static class

- (1) We can have a class inside a class which is known as inner class.
- (2) Inner class can be qualified with the keyword static.

Code :

```
class Example
{
    static class Test
    {
        public static String country = "India";
    }
}

public class Hello
{
    public static void main (String args[])
    {
        System.out.println (Example.Test.country);
    }
}
```

3

Date
30 Oct 2020

primitive data type → int, char, float

URBAN
EDGE

Wrapper Classes in Java

JAVA is 99% OOP

- JAVA is an ~~OO~~ Object oriented programming & as said everything in java is object.

Wrapper Classes

- As a solution to the problem of not 100% OOP, Java allows you to include the primitives in the family of objects by using what are called wrapper class.
- There is a wrapper class for every primitive data type in java.
- This class encapsulates a single value for the primitive data type.
- For instance the wrapper class for int is Integer, for float is Float & so on.

Boolean	→	Boolean
byte	→	Byte
char	→	Character
short	→	Short
int	→	Integer
long	→	Long
float	→	Float
double	→	Double

Useful methods of wrapper class

(1)

valueOf()

- Static method
- Return Object reference of relative wrapper class.

(2)

parseXxx()

Xxx = primitive data type

- Static method
- Xxx can be replaced by any primitive type.
- It returns Xxx type value.

(3)

~~XxxValue()~~ xxxValue()

- Instance method of wrapper class.
- Returns corresponding primitive type.

Code Example

```
public class Example  
{
```

```
    public static void main(String[] args)  
    {
```

```
        Integer i1 = Integer.valueOf("101011", 2);
```

```
        Double d1 = Double.valueOf("3.14");
```

```
        int a = Integer.parseInt("123");
```

```
        double b = Double.parseDouble("13.45");
```

```
        int c = i1.intValue();
```

```
    }
```

```
}
```

Command Line ArgumentsAccess Modifiers

- ⇒ Java supports four categories of accessibility rules
- private
 - protected
 - public
 - default
- ⇒ Modifiers can be used for class, member variable & member methods.

With Class

- (1) Class is divided into outer class & inner class.
- (2) For outer class, there can be only two possibilities, either class is a public class or just a class which means it is of default type.
- (3) For inner class any among four access modifiers can be used

Code:

```
public class Example //outer class
{
    class Dummy // inner class
    {
    }
}
```

Remember

- (1) There can be only one public class in a single java file.
- (2) The name of the java file must be the same as the name of the public class.
- (3) Only public class can be accessed directly from outside the package.

Lec 15

Initialization Block In Java

There are two types of Initialization blocks

→ Instance Initialization block

→ Static Initialization block

Instance Initialization Block

```
public class Test
{
```

```
    private int x;
```

```
    {
```

```
        System.out.println("Initialization Block: x=" + x);
```

```
        x = 5;
```

```
    }
```

```
    public Test()
```

```
    {
```

```
        System.out.println("Constructor: x=" + x);
```

```
    }
```

```
    public static void (String args[])
```

```
    {
```

```
        Test t1 = new Test();
```

```
        Test t2 = new Test();
```

```
    }
```

```
}
```

Instance
Initialization
block

It comes here

Note: If there are more than one Instance Initialization blocks, then they all merge up & add to be in the first of constructor. If no constructor, then only Instance Initialization blocks.

Important points

- (1) An instance initializer or Initialization block declared in a class is executed when an instance of the class is created.
- (2) return keyword cannot be used in Initialization block.
- (3) Instance initializers are permitted to refer to the current object via the keyword `this` & to use the keyword `super`.

Static Initialization Block

```
public class Test  
{
```

```
    private static int k;  
    static  
    {
```

```
        System.out.println("Static Initialization Block  
                             : k = " + k);
```

```
        k = 10;
```

```
    }
```

→ only static things can be used

```
    public static void main (String args[])
```

```
    {
```

```
        new Test();
```

```
    }
```

```
}
```

Note :

Static Initialization is automatically called when an object is created or ~~or~~ we write calling any static or instance method of that class. For ex:

Test.display();

Important Points

- (1) A static initializer declared in a class is executed when the class is initialized.
- (2) Static initializers are used to initialize the static variables of the class.
- (3) volatile keyword can not be used in static initialization block.
- (4) this or super can not be used in static block.

Method Overriding

```
class A
{
    public void f1(int n)
    {
        System.out.println("Class A");
    }
}
```

```
class B extends A
{
    public void f1(int n)
    {
        System.out.println("Class B");
    }
}
```

```
public class Example1
{
```

```
public static void main (String args[])
{
```

```
    B obj = new B(1);
    obj.f1(5);
}
```

Final Keyword

- (1) final instance variable
- (2) final static variable
- (3) final local variable
- (4) final class
- (5) final methods

Three ways
of initialising
final variable

Final Instance Variable

- during declaration
- Inside initialization block
- Inside constructor

```
public class Example
{
```

```
    private final int n; // final instance member var.
```

```
    Example()
    {
```

```
        n = 5;
    }
```

```
    public static void main (String args[])
    {
```

```
        Example e1 = new Example();
    }
```

Note :

- (1) A java variable can be declared using the keyword final. Then the final variable can be assigned only once.
- (2) A variable that is declared as final and not initialized is called a blank final variable. A blank final variable forces either the constructor to initialize it or initialization block to do this job.

Final Static Variable

```
public class Example
{
```

```
    private final int n;
```

```
    private final static int y;
```

```
    Example ()
```

```
    {
```

```
        n = 8;
```

```
    } static { y = 4; }
```

```
    psvm ( )
```

```
    {
```

```
        Example e1 = new Example ();
```

```
    }
```

```
}
```

final word can be written before static & after static also

Note :

- (1) Static member variable when qualified with final keyword, it becomes blank until initialised.
- (2) Final static variable can be initialized during declaration or within the static blocks.

note: Local variables are blank variable, & final static local variable also.

URBAN
EDGE

Final Local Variable

```
public class Example
```

```
{
```

```
    public void fun()
```

```
    {
```

```
        final int k; // final local variable
```

```
    }
```

```
}
```

Note:

- (1) Local variables that are final must be initialised before it's use, but you should remember this rule is applicable to non final variable too.
- (2) Once they are initialized, can not be altered.

Final class

Java class declared as final cannot be extended. Restricting inheritance!

```
final class Dummy()
```

```
{
```

```
}
```

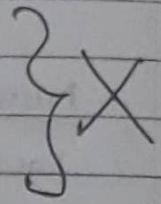
Final Method

Methods declared as final can not be overridden.

```
class Dummy  
{  
    public void someMethod  
}
```

```
public class Dummy
{
    final
    public void someMethod()
    {
    }
}
```

```
class MoreDummy extends Dummy
{
    final
    public void someMethod
    {
    }
}
```



this keyword

class Box

{

private int l, b, h;

public void setDimension(int ^ll, int ^bb, int ^hh)

{

this.l = l;

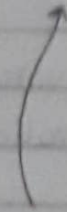
this.b = b;

this.h = h;

}

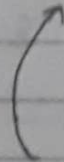
}

instance member method



without this ambiguity will occur

static member method



public class Example

{

public static void main(String args[])

{

Box b1 = new Box();

b1.setDimension(12, 10, 5);

}

}

Imp. Note:

- (1) The this object reference is a local variable in instance member methods referring the caller object.
- (2) this keyword is used as a reference to the current object which is an instance of the current class.
- (3) The this reference to the current object is useful in situations where a local variable hides, or shadows, a field with the same name.

```
class Bon
```

```
{
```

```
    private int l, b, h;
```

```
    public void setDimension(int l, int b, int h)
```

```
{
```

```
        this.l = l;
```

```
        this.b = b;
```

```
        this.h = h;
```

```
    }
```

```
    public void sendBon()
```

```
{
```

```
        Giftaker gt = new GiftTaker();
```

```
        gt.acceptGift(this);
```

```
    }
```

```
}
```

```
public class Example
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
        Bon b1 = new Bon();
```

```
        b1.setDimension(12, 10, 5);
```

```
        b1.sendBon();
```

```
    }
```

```
}
```

Note:

- (1) If a method needs to pass the current object to another method, it can do so using this reference.
- (2) Note that this reference cannot be used in a static context, as static code is not executed in the context of any object.

Q Is this can be used in constructor?

Ans Yes, becoz constructor is instance method

Q Is this can be used inside main?

Ans No, becoz main is static method

Super keyword

class A

```
{
    public void f1()
    {
    }
}
```

class B extends A

```
{
    public void f1()
    {
```

super.f1();

}

}

class Example

{

public static void main (String args[])

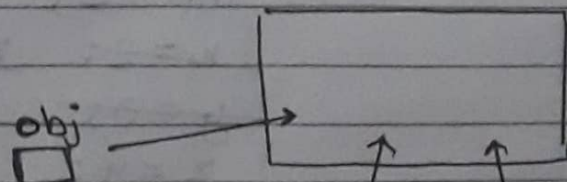
{

B obj = new B();

obj.f1();

}

}



Local
inside member

this

super

B this = new B();

A this = new A();

Note

- (1) In Inheritance, subclass object when call an instance member method of subclass only, method contains implicit reference variable `this` and `super` both referring to the current object (object of subclass).
- (2) The only difference in `this` and `super` is
 (a) `this` reference is of subclass type
 (b) `super` reference is of superclass type

Use of super keyword

- (1) If your method overrides one of its superclass's methods, you can invoke the superclass version of the method through the use of the keyword `super`.
- (2) It avoids name conflict (ambiguity) between members of superclass & subclass

```
class A {
    int xz;
}
```

```
class B extends A
```

```
{
```

```
    z
```

```
    int yz;

```

```
private void
```

```
    public void f2()
```

```
{
```

```
    int z;
```

```
    x = 2; z = 2; super.z = 2;
```

```
    y = 3; z = 3; this.z = 3;
```

```
    z = 4;
```

```
}
```

```
class Example
```

```
{
```

```
    public static void main (String args[])
```

```
    {
```

```
        B obj = new B();
```

```
        obj.t2();
```

```
    }
```

```
}
```

Constructors in Java

static members in Inheritance

```
class Parent
{
    public static void f1()
    {
        System.out.println("Hello");
    }
}
```

```
class Child extends Parent
{
}
```

```
public class Example
{
    public static void main(String args[])
    {
        Child.f1();
    }
}
```

Points

- (1) A class C inherits from its direct superclass all concrete methods m (both static & instance) of the superclass.
- (2) No method declared in C has to same signature.

```
class Parent
```

```
{
```

```
    public static void f1()
```

```
    {
```

```
        System.out.println("Hi, I am from Parent");
```

```
    }
```

```
}
```

```
class Child extends Parent
```

```
{
```

```
    public static void f1()
```

```
    {
```

```
        System.out.println("Hi, I am from child");
```

```
    }
```

```
}
```

```
public class Example
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        Child.f1();
```

```
    }
```

```
}
```

Method Hiding

If subclass has a method m with the same signature as of the method present in the super class, then method m hides the method of superclass.

Remember

- (1) It is compile-time error if a static method hides an instance method.

- (2) It is a compile-time error if an instance method overrides a static method.

Static member variables

```
class Parent
```

```
{
```

```
    static int y = 4;
```

```
}
```

```
class Child extends Parent
```

```
{
```

```
    static
```

```
    { y = 5; }
```

```
}
```

```
public class Example
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        System.out.println("y=" + Child.y); //y=4
```

```
    }
```

```
}
```

Note: static member variables do not inherit
but static member method do

Constructor in Inheritance

(1) Constructors are not inherited by class

Q What happens when object of subclass created?

Q What is the role of constructor?

```
class A
```

```
{
```

```
    int a;
```

```
    public A()
```

```
{
```

```
        System.out.println("A");
```

```
}
```

```
}
```

Output

A

B

```
class B extends A
```

```
{
```

```
    int b;
```

```
    public B()
```

```
{
```

```
        super();
```

```
        System.out.println("B");
```

```
}
```

```
}
```

→ It should be write in first line of constructor.

→ If you will not write then compiler will write.

```
class Example
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
        B obj = new B();
```

```
}
```

```
}
```

Point

sub class's constructor invokes constructor of super class.

Code

```
class A
```

```
{
```

```
    int a;
```

```
    public A(int n)
```

```
{
```

```
        a = n;
```

```
        System.out.println("A");
```

```
    }
```

```
}
```

```
class B extends A
```

```
{
```

```
    int b;
```

```
    public B()
```

```
{
```

```
        super(4);
```

```
        System.out.println("B");
```

```
    }
```

```
}
```

```
class Example
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
        B obj = new B();
```

```
    }
```

```
}
```

Points

- (1) Explicit call to the super class constructor from sub class's constructor can be made using `super()`.
- (2) You can write a subclass constructor that invokes the constructor of the superclass, either implicitly or by using the keyword `super`.

Scenarios

- (1) Implicit constructors in superclass & subclass.
- (2) Implicit constructor in subclass and explicit constructor in superclass.
- (3) Implicit constructor in superclass & explicit constructor in subclass.
- (4) Explicit constructor in superclass & subclass.

Constructor chaining in JAVA

class A

{

public A()

{

System.out.println("A");

}

}

class B extends A

{

public B()

{

this(4); $\longrightarrow$ It will call constructor of same class with 1 argument

System.out.println("B 1");

}

int x
public B()

{

System.out.println("B 2");

}

}

class Example

{

public static void main(String args[])

{

B obj = new B();

}

}

Output

A

B2

B1

Points

- (1) Constructor can call other constructor of the same class or super class.
- (2) Constructor call from a constructor must be the first step. (call should appear in the first line).
- (3) Series of invocation of constructor is known as constructor chaining.

Arrays in Java

```
int arr[5]; // error
```

```
int arr[5];
```

```
int arr[];
```

reference variable of array

So, proper syntax of making array

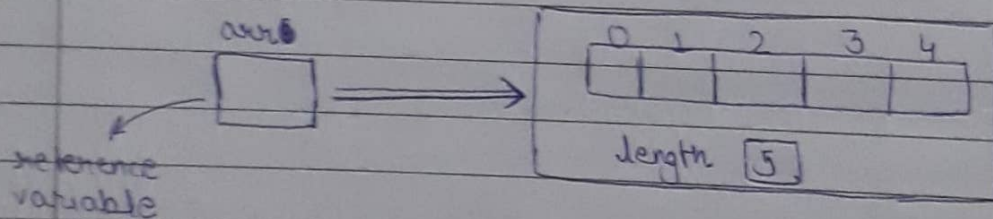
It is similar like making object

```
int arr[] = new int[5];
```

or we can change position of brackets

```
int [5]
```

```
int[] arr = new int[5];
```

Memory allocation of array in Java

$arr[0] = \text{value}$

$arr.length = 5$

Initialization at the time of declaration

```
int arr[] = new int[] { 2, 4, 6, 8, 10 };
```

or

```
int arr[] = { 2, 4, 6, 8, 10 };
```

Some common mistakes & errors

- (1) `int arr[] = new int[2] {2, 4, 6, 8, 10};`
- (2) `int arr[] = new int[5] {2, 4, 6, 8, 10};`
 // you can not mention size & values together

Note

Array is not blank means If we won't initialize array then by default value 0 will be put inside 0.

```
int arr[] = new int[3]
```

```
System.out.println(" arr[0] " + arr[0]);
```

⊙ // Output: arr[0] = 0

Find Error in Program?

```
public class ArrayExample
{
```

```
    public static void main(String args[])
    {
```

```
        int arr[];
```

```
        arr[0] = 25;
```

```
        arr[1] = 50;
```

```
    }
```

```
}
```

→ we have not allocated for the array.
So, it will act as local blank variable.

Initialize after Declaration

```
public class ArrayExample
{
```

```
    public static void main(String args[])
    {
```

```
        int arr[];
```

```
        arr = new int[3];
```

```
        arr[0] = 10; arr[1] = 20; arr[2] = 30;
```

```
    }
```

```
}
```

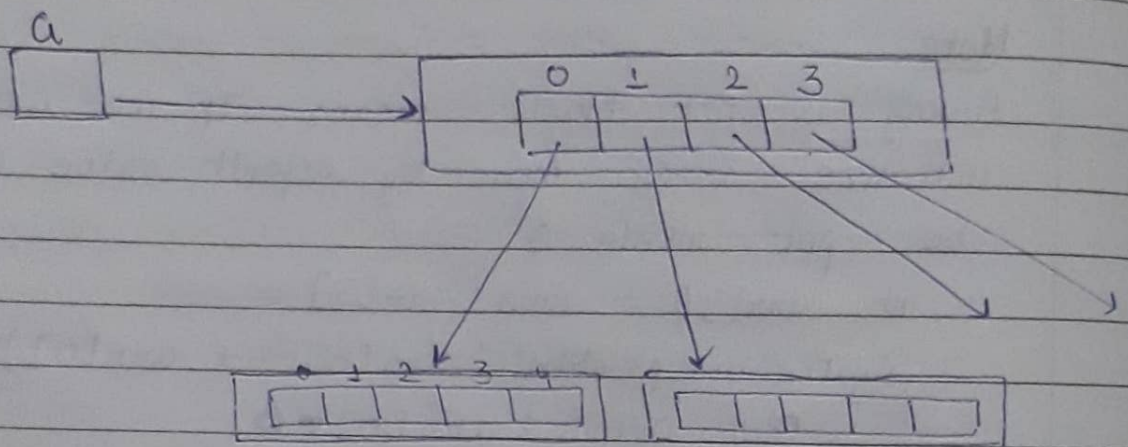
Two Dimensional Array

```
int arr[][];
```

```
int [][]arr;
```

So,

```
int [][]a = new int[4][5];
```



Note

- (1) `int a[][] = new int[][];` // error
- (2) `int a[][] = new int[][5];` // error
- (3) `int a[][] = new int[4][];` // ~~error~~ ✓

Initializing 2D array at declaration

```
int a[][] = new int[][] { {3,4}, {5,6}, {7,8} };
```

Find Error

```
public class Example
{
```

```
    public static void main(String args[])
    {
```

```
        int[][] a = new int[4][2];
```

```
        System.out.println("a[0][0]" + a[0][0]);
```

3

3

Error: Null pointer Exception

Use of length property

```
public class Example1
{
```

```
    public static void main(String args[])
    {
```

```
        int [][]a = new int[4][2];
```

```
        System.out.println("Length=" + a.length);
```

3

3

Output: 4

Find output

```
public class Example1
{
```

```
    public static void main(String args[])
    {
```

```
        int [][]a = new int[4][2];
```

```
        System.out.println("Length=" + a[0].length);
```

3

3

Example : User Input

```
import java.util.*;
```

```
class ArrayExample {
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        int arr[] = new int[5];
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        for(int i=0; i<5; i++)
```

```
        {
```

```
            arr[i] = scanner.nextInt();
```

```
        }
```

```
        for(int i=0; i<5; i++)
```

```
        {
```

```
            System.out.println("arr[" + i + "] = " + arr[i]);
```

```
        }
```

```
    }
```

```
}
```

2D array with different sizes

```
public class Example
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        int a[][] = new int[4][];
```

```
        a[0] = new int[5];
```

```
        a[1] = new int[3];
```

```
        System.out.println("Length=" + a[1].length)
```

```
    }
```

```
}
```